

Firestore Hosting Deployment with Workload Identity Federation

Technical Reference and Replication Guide

Andres Gonzalez Ortega • May 3, 2026

Abstract. This document describes the complete setup of automated CI/CD deployment for a Next.js static dashboard to Firestore Hosting, using Google Cloud Workload Identity Federation (WIF) for keyless authentication from GitHub Actions. It covers the GCP infrastructure (service account, IAM roles, WIF pool and provider), the GitHub Actions workflow, the security posture of the configuration, and step-by-step replication instructions. The approach eliminates long-lived credentials entirely: no JSON key files, no CI tokens—only short-lived, automatically rotated access tokens scoped to a single repository and a single GCP role.

Contents

Reading Guide	3
1 Introduction	3
2 Background and Notation	3
3 Architecture	4
3.1 Data and Build Pipeline	4
3.2 Authentication Flow	4
4 GCP Infrastructure	5
4.1 Service Account	5
4.2 IAM Roles	5
4.3 Workload Identity Pool and Provider	5
4.4 IAM Binding	5
5 GitHub Actions Workflow	6
5.1 Triggers	6
5.2 Permissions	6
5.3 Steps	7
5.4 Credential Configuration	7
6 Security Audit	7
6.1 Comparison of Authentication Methods	7
7 Operational Procedures	7
7.1 Automated Deployment	8
7.2 Manual Deployment	8
7.3 Rollback	8
7.4 Data Refresh	9

8	Replication Guide	9
8.1	Prerequisites	9
8.2	Step 1: Create Service Account	9
8.3	Step 2: Grant IAM Roles	9
8.4	Step 3: Create WIF Pool (if none exists)	9
8.5	Step 4: Create GitHub OIDC Provider	10
8.6	Step 5: Bind WIF to Service Account	10
8.7	Step 6: Store Credential Config as GitHub Secret	10
8.8	Step 7: Create Workflow File	10
8.9	Step 8: Commit and Verify	10
9	Limitations	11
	Appendix: Project Context	11

Reading Guide

This document uses three numbered theorem-style environments. Their meaning is fixed for the rest of the document; the environment name and its number identify the kind of statement, so no further explanatory titles are added.

- **Definition** — a formal statement that fixes the meaning of a term. The numbered label is the canonical reference for that term.
- **Proposition** — a load-bearing claim used in subsequent reasoning. Numbered in the same sequence as definitions so cross-references read naturally.
- **Remark** — an aside that clarifies an edge case, an alternate notation, or an implication that does not justify a full proposition.

After this guide, the environments appear without descriptive labels; only the standard mathematical convention of placing the defined term in the optional argument (e.g. `\begin{definition}[Hazard rate]`) is preserved.

1 Introduction

The `learning-posgre` project is a flight analytics platform built around a PostgreSQL 16 demo database containing 5.74 million flight records. The project includes SQL analysis scripts, a BigQuery migration layer, an ETL pipeline, and a Next.js 16 dashboard deployed as a static site to Firebase Hosting.

Deploying from a local machine via `firebase deploy` works but introduces two risks: (1) it depends on a human remembering to deploy after changes, and (2) it requires the deployer's personal Firebase credentials. The standard solution is CI/CD via GitHub Actions, but the authentication mechanism matters.

Three approaches exist for authenticating GitHub Actions to GCP:

- (1) **Firebase CI token**: a long-lived OAuth refresh token generated by `firebase login:ci`. Never expires. Full permissions of the issuing user account.
- (2) **Service account JSON key**: a long-lived private key file stored as a GitHub secret. Must be manually rotated.
- (3) **Workload Identity Federation (WIF)**: a keyless mechanism where GitHub's OIDC tokens are exchanged for short-lived GCP access tokens. No persistent credentials are stored anywhere.

This document describes option (3). WIF was chosen because it eliminates credential management entirely and provides the smallest blast radius if any component is compromised.

2 Background and Notation

Definition 2.1 (Workload Identity Federation). A GCP mechanism that allows external identities (e.g., GitHub Actions runners) to impersonate GCP service accounts without storing private keys. It works by establishing a trust relationship between an external OIDC identity provider and a GCP Workload Identity Pool.

Definition 2.2 (OIDC Token). An OpenID Connect JSON Web Token (JWT) issued by GitHub's token endpoint (`token.actions.githubusercontent.com`) during each workflow run. The token contains claims identifying the repository, branch, workflow, and actor. It is short-lived (valid for approximately 15 minutes).

Table 1: Notation and identifiers used throughout this document.

Symbol / Identifier	Meaning	Value
PROJECT_ID	GCP project identifier	project-ad7a5be2-a1c7-4510-82d
PROJECT_NUMBER	GCP project number	451451662791
SA_EMAIL	Service account email	firebase-hosting-deploy@{...}.iam...
REPO	GitHub repository	GonorAndres/learning-posgre
POOL	WIF pool name	github-pool
PROVIDER	WIF provider name	github-provider

Definition 2.3 (Workload Identity Pool). A GCP resource that acts as a trust boundary for external identities. A pool contains one or more providers and defines attribute conditions that restrict which external tokens are accepted.

Definition 2.4 (Static Export). A Next.js build mode (`output: "export"`) that generates plain HTML, CSS, and JavaScript files with no server-side runtime. All data must be available at build time. The output directory (`dashboard/out/`) can be served by any static hosting provider.

3 Architecture

3.1 Data and Build Pipeline

The deployment pipeline has three stages:

Stage 1: Data generation. The script `dashboard/scripts/prepare-data.py` connects to PostgreSQL, runs analytical queries, and writes 10 JSON files to `dashboard/data/`. These files contain pre-computed metrics for delays, revenue, utilization, geospatial routes, and PostgreSQL internals.

Stage 2: Static build. Running `npx next build` in the `dashboard/` directory compiles TypeScript, bundles React components, and generates static HTML for each route into `dashboard/out/`. The JSON data files are embedded via static imports.

Stage 3: Deployment. The contents of `dashboard/out/` are uploaded to Firebase Hosting, which distributes them across Google’s global CDN (200+ edge locations). A new “release” is created, and the previous release remains available for instant rollback.

3.2 Authentication Flow

The WIF authentication flow during a GitHub Actions run proceeds as follows:

1. The GitHub Actions runner requests an OIDC token from GitHub’s token endpoint. This requires the workflow to declare `permissions: id-token: write`.
2. GitHub issues a JWT containing claims: `repository`, `repository_owner`, `actor`, `ref`, and others.
3. The `google-github-actions/auth@v2` action sends this JWT to GCP’s Security Token Service (STS) at `sts.googleapis.com/v1/token`.
4. STS validates the JWT against the WIF pool and provider configuration:
 - Issuer matches `token.actions.githubusercontent.com`.
 - Attribute condition passes: `assertion.repository_owner == 'GonorAndres'`.

5. STS checks the IAM binding on the target service account: only principals matching `attribute.repository/GonorAndres/learning-posgre` have `roles/iam.workloadIdentityUser`.
6. If all checks pass, STS returns a short-lived access token (valid approximately 15 minutes) that can act as the service account.
7. The `FirestoreExtended/action-hosting-deploy@v0` action uses this token to upload `dashboard/out/` to Firestore Hosting.

Proposition 3.1. No persistent credentials are stored at any point in this flow. The GitHub OIDC token is generated per-run and expires within minutes. The GCP access token is generated per-run and expires within 15 minutes. The `FIREBASE_SERVICE_ACCOUNT` GitHub secret contains only a credential configuration file (URLs and resource identifiers), not a private key.

4 GCP Infrastructure

4.1 Service Account

A dedicated service account was created with the principle of least privilege:

Table 2: Service account configuration.

Property	Value
Name	<code>firebase-hosting-deploy</code>
Email	<code>firebase-hosting-deploy@project-ad7a5be2-a1c7-4510-82d.iam.gserviceaccount.com</code>
Display name	Firestore Hosting GitHub Actions

4.2 IAM Roles

Table 3: IAM roles granted to the service account.

Role	Purpose
<code>roles/firebasehosting.admin</code>	Create and update Firestore Hosting releases. This is the only role needed for deployment.
<code>roles/iam.serviceAccountTokenCreator</code>	Required for WIF token exchange. Allows the STS to generate access tokens on behalf of this service account.

Remark 4.1. The service account has no access to Cloud Storage, BigQuery, Compute Engine, or any other GCP service. If the credentials were somehow compromised, the attacker could only deploy static files to this specific Firestore Hosting site.

4.3 Workload Identity Pool and Provider

An existing pool (`github-pool`) was reused. It was originally created for Cloud Run deployments from other repositories under the same GCP project.

4.4 IAM Binding

The binding that connects WIF to the service account is scoped to a single repository:

Table 4: WIF pool and provider configuration.

Property	Value
Pool ID	projects/451451662791/locations/global/workloadIdentityPools/github
Provider ID	github-provider
OIDC issuer	https://token.actions.githubusercontent.com
Attribute condition	assertion.repository_owner == 'GonorAndres'
Attribute mapping	google.subject = assertion.sub, attribute.repository = assertion.repository

```
principalSet://iam.googleapis.com/projects/451451662791
/locations/global/workloadIdentityPools/github-pool
/attribute.repository/GonorAndres/learning-posgre
```

Remark 4.2. The pool-level attribute condition (`repository_owner == 'GonorAndres'`) is a coarse filter: it accepts tokens from any repository under the `GonorAndres` GitHub account. The fine-grained security comes from the IAM binding above, which restricts impersonation to a specific repository. This is a layered defense: even if a new repository were created under the same GitHub account, it could not impersonate this service account without an explicit IAM binding.

5 GitHub Actions Workflow

The workflow file is located at `.github/workflows/firebase-deploy.yml`.

5.1 Triggers

Table 5: Workflow trigger configuration.

Trigger	Condition
push	Branch <code>main</code> , paths: <code>dashboard/**</code> , <code>firebase.json</code>
workflow_dispatch	Manual trigger from GitHub Actions UI

The path filter ensures that commits touching only SQL scripts, documentation, or pipeline code do not trigger a rebuild of the dashboard.

5.2 Permissions

Table 6: Workflow-level GITHUB_TOKEN permissions.

Permission	Purpose
<code>contents: read</code>	Required for <code>actions/checkout</code> to clone the repository.
<code>id-token: write</code>	Required for GitHub to issue the OIDC token consumed by the WIF exchange. Without this permission, the <code>google-github-actions/auth</code> step fails silently.

Table 7: Workflow steps in execution order.

Step	Action / Command	Purpose
1	actions/checkout@v4	Clone repository
2	actions/setup-node@v4	Install Node.js 20, restore npm cache
3	npm ci	Install dependencies from lockfile
4	npx next build	Generate static export to dashboard/out/
5	google-github-actions/auth@v2	Exchange OIDC token for GCP access token via WIF
6	action-hosting-deploy@v0	Deploy to Firebase Hosting

5.3 Steps

5.4 Credential Configuration

The FIREBASE_SERVICE_ACCOUNT GitHub secret contains a JSON credential configuration file. This file is *not* a private key. Its structure:

```
{
  "type": "external_account",
  "audience": "://iam.googleapis.com/projects/451451662791
    /locations/global/workloadIdentityPools
    /github-pool/providers/github-provider",
  "subject_token_type":
    "urn:ietf:params:oauth:token-type:jwt",
  "token_url":
    "https://sts.googleapis.com/v1/token",
  "credential_source": {
    "url":
      "https://token.actions.githubusercontent.com",
    "format": {
      "type": "json",
      "subject_token_field_name": "value"
    }
  },
  "service_account_impersonation_url":
    "https://iamcredentials.googleapis.com/v1
    /projects/-/serviceAccounts/
    firebase-hosting-deploy@...
    :generateAccessToken"
}
```

Remark 5.1. This JSON tells the Google Auth library *where* to exchange tokens, not *how* to authenticate. The actual authentication comes from the OIDC token that GitHub issues at runtime. Without a valid GitHub Actions environment, this configuration file is inert.

6 Security Audit

6.1 Comparison of Authentication Methods

7 Operational Procedures

Table 8: Security checklist for the deployment configuration.

Check	Result
API keys, passwords, or tokens in committed files	None
JSON key files for service account	None (keyless)
Workflow permissions scope	Least-privilege
WIF binding scope	Single repository
Service account IAM roles	<code>firebasehosting.admin</code> only
Workflow dispatch input injection	No inputs defined
Actions version pinning	Major versions
GitHub secret content	Non-sensitive config

Table 9: Security properties across three authentication approaches.

Property	CI Token	JSON Key	WIF
Credential type	Long-lived OAuth refresh token	Long-lived private key	Short-lived (15 min) auto-generated
Rotation	Manual revoke	Manual rotation	Automatic
Blast radius if leaked	Full Firebase access as user	Full SA access until revoked	Useless after ~15 min
Stored as GitHub Secret	Sensitive	Sensitive	Non-sensitive config
Permission scope	User’s full permissions	SA’s full permissions	SA’s permissions (hosting only)

7.1 Automated Deployment

Push any change to `dashboard/**` or `firebase.json` on the `main` branch. GitHub Actions builds the static site and deploys it automatically. No manual intervention required.

7.2 Manual Deployment

Two options:

- GitHub Actions UI:** navigate to the Actions tab, select “Deploy to Firebase Hosting”, click “Run workflow”.
- Local CLI:**

```
cd dashboard
python scripts/prepare-data.py # if data changed
npm run build
cd ..
firebase deploy --only hosting
```

7.3 Rollback

Firebase Hosting retains previous releases. To revert:

```
firebase hosting:rollback
```

Alternatively, use the Firebase Console to select a specific previous release.

7.4 Data Refresh

If the underlying PostgreSQL data changes:

1. Run `python scripts/prepare-data.py` in the `dashboard/` directory to regenerate the 10 JSON files.
2. Commit the updated JSON files to `main`.
3. The push triggers an automatic rebuild and deploy.

8 Replication Guide

This section provides the complete sequence of commands to reproduce this setup from scratch on any GCP project with a GitHub repository.

8.1 Prerequisites

- `gcloud` CLI installed and authenticated
- `gh` CLI installed and authenticated
- `firebase` CLI installed and authenticated
- A GitHub repository with a static site to deploy
- Firebase Hosting initialized (`firebase init hosting`)

8.2 Step 1: Create Service Account

```
PROJECT_ID="<your-project-id>"
SA_NAME="firebase-hosting-deploy"

gcloud iam service-accounts create "$SA_NAME" \
  --display-name="Firebase Hosting GitHub Actions" \
  --project="$PROJECT_ID"
```

8.3 Step 2: Grant IAM Roles

```
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/firebasehosting.admin" \
  --condition=None --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/iam.serviceAccountTokenCreator" \
  --condition=None --quiet
```

8.4 Step 3: Create WIF Pool (if none exists)

```
gcloud iam workload-identity-pools create github-pool \
  --location="global" \
  --display-name="GitHub Actions Pool" \
  --project="$PROJECT_ID"
```

8.5 Step 4: Create GitHub OIDC Provider

```
GITHUB_USER="<your-github-username>"

gcloud iam workload-identity-pools providers \
  create-oidc github-provider \
  --location="global" \
  --workload-identity-pool="github-pool" \
  --display-name="GitHub Provider" \
  --issuer-uri=\
    "https://token.actions.githubusercontent.com" \
  --attribute-mapping=\
    "google.subject=assertion.sub,\
    attribute.repository=assertion.repository" \
  --attribute-condition=\
    "assertion.repository_owner==\
    '${GITHUB_USER}'" \
  --project="$PROJECT_ID"
```

8.6 Step 5: Bind WIF to Service Account

```
PROJECT_NUMBER=$(gcloud projects describe \
  "$PROJECT_ID" --format="value(projectNumber)")
REPO="${GITHUB_USER}/<your-repo>"

gcloud iam service-accounts add-iam-policy-binding \
  "$SA_EMAIL" \
  --project="$PROJECT_ID" \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/\
    projects/${PROJECT_NUMBER}/locations/global/\
    workloadIdentityPools/github-pool/\
    attribute.repository/${REPO}"
```

8.7 Step 6: Store Credential Config as GitHub Secret

Generate the WIF credential configuration JSON (see Section 5.4 for the structure) and store it:

```
gh secret set FIREBASE_SERVICE_ACCOUNT \
  --repo="${REPO}" < wif_cred.json
```

8.8 Step 7: Create Workflow File

Copy the workflow YAML (Section 5) to `.github/workflows/firebase-deploy.yml`, replacing the `workload_identity_provider` and `service_account` values with your project-specific identifiers.

8.9 Step 8: Commit and Verify

```
git add .github/workflows/firebase-deploy.yml
git commit -m "Add Firebase deploy with WIF auth"
git push origin main
```

Monitor the first run in the GitHub Actions tab to confirm successful authentication and deployment.

9 Limitations

- **Data freshness.** The dashboard shows pre-computed data. If the PostgreSQL database is updated, `prepare-data.py` must be re-run manually and the JSON files committed. The CI/CD pipeline does not connect to the database.
- **Action version pinning.** The workflow pins to major versions (`@v4`, `@v2`, `@v0`). For maximum supply-chain security, these could be pinned to specific commit SHAs, at the cost of manual updates when new versions are released.
- **Single environment.** There is no staging/production split. Every push to `main` deploys directly to the live site. For a portfolio project this is acceptable; for production use, Firebase preview channels should be added.
- **No preview deploys on PRs.** The current workflow only deploys on merge to `main`. Adding a second job triggered on `pull_request` with a non-live `channelId` would enable preview URLs for each PR.

Appendix: Project Context

This setup was implemented for the `learning-posgre` project, a flight analytics platform hosted at <https://github.com/GonorAndres/learning-posgre>. The Firestore Hosting site is available at <https://project-ad7a5be2-a1c7-4510-82d.web.app>.

Table 10: File inventory for the deployment configuration.

Artifact	Location
GitHub Actions workflow	<code>.github/workflows/firebase-deploy.yml</code>
Firebase config	<code>firebase.json</code>
Firebase project alias	<code>.firebaserc</code>
Next.js config (static export)	<code>dashboard/next.config.ts</code>
Data generation script	<code>dashboard/scripts/prepare-data.py</code>
Pre-computed JSON data	<code>dashboard/data/*.json</code> (10 files)
Static build output	<code>dashboard/out/</code>